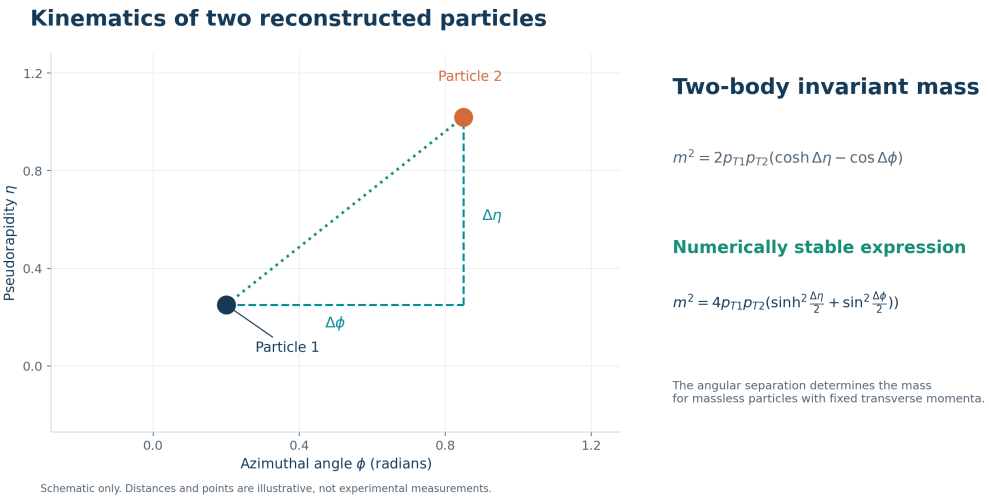
Reusable Mathematical Execution for Collider Research

SCIENTIFIC WHITE PAPER | IMPLEMENTATION AND VERIFIED RESULTS

An experimentally verified C++ demonstrator using the original paceval Linux runtime

paceval UG · Munich · 10 November 2026

Research outcome. The demonstrator executes real, persistent paceval graphs for collider-inspired kinematic quantities, synthetic event weights, and repeated systematic variations. Seven independent runs demonstrate a **1.21×** median speed factor for consolidating three paceval graphs into one multi-graph API call, **1.10×** for a fused mass–weight computation, and **1.05×** for flattening five variations into larger batches. All **24 of 24** tested TINC intervals enclosed independent 110-decimal-digit reference values.



Kinematic context: two reconstructed particles

Figure 1. Schematic angular separation in pseudorapidity and azimuth. This is not a CERN event display.

1. Scientific objective

This paper reports the implementation and experimental verification of a focused study of scalar kinematic kernels, repeatable event weights, and reusable mathematical execution graphs in collider-

inspired analysis. A fully reproducible C++ implementation uses the original paceval Linux runtime, an independently written native reference, seven-run benchmark measurements, 110-digit high-precision checks, and detailed graphics. The study is distributed as a compact GitHub-ready research package (`README.md`), with the main source (`src/collider.cpp`) and retained measurement samples (`examples/benchmark_runs.csv`).

The input events are deterministic and synthetic, and the event-weight function is explicitly **not a calibrated CMS detector model**. The demonstrator addresses the computational layer of a potential existing analysis, not track reconstruction, detector simulation or measurements of CERN collision events.

2. What was implemented with paceval

The program loads the documented `pacevalLibrary_*` symbols and uses `pacevalLibrary_CreateComputation` to create **four persistent evaluation graphs** and **two TINC graphs**.

Handle	Mathematical operation and role
G1	Direct squared invariant mass of a massless particle pair
G2	Algebraically equivalent, numerically stable squared invariant mass
G3	Parametric synthetic event weight with eight inputs
G4	A single fused expression for stable invariant mass squared × synthetic weight
T1, T2	The two mass formulas with Trusted Interval Computing enabled

The two massless-particle formulas, with transverse momenta in GeV, are:

- Direct: $m^2 = 2\ pT1\ pT2\ [\cosh(\Delta\eta) - \cos(\Delta\varphi)]$
- Stable: $m^2 = 4\ pT1\ pT2\ [\sinh^2(\Delta\eta/2) + \sin^2(\Delta\varphi/2)]$

The intentionally synthetic weight is parameterized by `a, b, c, d`:

```
W = (1 + a*exp(-abs(deta))*cos(dphi)
       + b*ln(1+pt1/100)
       + c*sin(pt2/30)^2)
    * (1 + d*tanh((pt1+pt2-100)/60))
```

The corrected paceval expression uses `ln(...)` for the natural logarithm, matching the published formula language. Graphs are constructed once and the input variable values are supplied repeatedly. `pacevalLibrary_dGetComputationResultExt` evaluates event batches, while `pacevalLibrary_dGetMultipleComputationsResultsExt` returns three observables through a consolidated invocation. The weight graph is reused across five systematic parameter variations, both sequentially and with flattened inputs. The TINC-enabled calls return value, lower limit and upper limit.

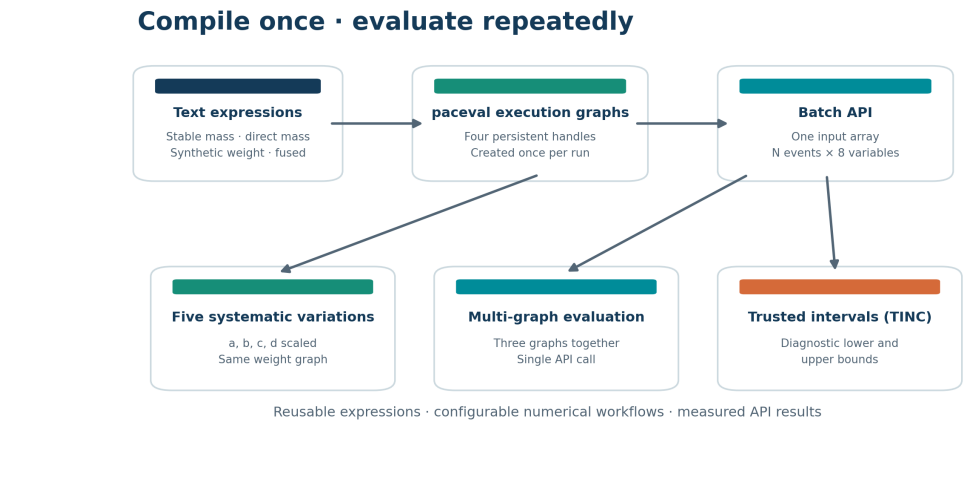


Figure 2. Actual evaluated execution paths implemented with the original paceval library.

3. Experimental environment and measurement protocol

The measurements ran in a **Debian GNU/Linux 13 (trixie)** environment, with the **Linux 6.18.57 kernel** on **x86-64** under **KVM virtualization**. The virtual machine exposed **five AMD EPYC 9V74 virtual CPUs**, although the container CPU quota observed in the testing environment was **four CPU-equivalents** (`cpu.max = 400000 100000`). The visible five virtual CPUs describe **available virtual hardware**, not five CPU workers used by the application: this C++ demonstration executes its graph-evaluation batches **sequentially in one process**.

The reference executable was compiled with **GCC 14.2**, **C++17**, and **-O3** optimization; compilation checks also passed with **Clang 17**. Evaluation used the original external paceval runtime `libpaceval_linux_sharedLIB.so`, supplied separately from this GitHub source repository, with SHA-256:

`acf63a87df0ee822b467edbca887393d31ec0ea03562e1abf802d2512a5bfb12`

The principal run contains **120,000** deterministic synthetic events generated with seed **20261110**, including approximately 20% small-angle cases. Batch size was **16,384** events per API invocation. Five systematic scale factors, from 0.8 to 1.2, modified four weight parameters while **reusing the same prebuilt graph**. Timings separately record construction, packing, and evaluation. Reported speed factors are the **medians of paired ratios from seven independent, fresh-process executions**, not ratios of pooled averages. The full paired observations appear in `examples/benchmark_runs.csv` (`examples/benchmark_runs.csv`), with summary statistics in `examples/benchmark_summary.csv` (`examples/benchmark_summary.csv`).

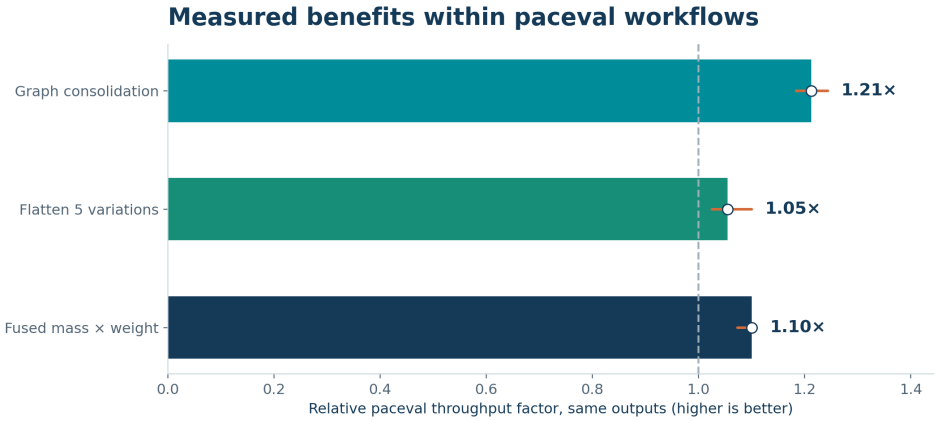
The measured platform and compiler are specified for reproducibility. Times are specific to this software, workload, virtual CPU allocation, and runtime build.

4. Measured benefits within paceval workflows

Alternative paceval execution strategy	Seven-run median	Verified contribution
Three separate graphs	Sum of three execution stages	Baseline for graph consolidation
Three graphs returned by one multi-graph call	**232.66 ms**	**1.21×** median factor vs three separate paceval stages
Fused stable mass² × event weight	**204.94 ms**	**1.10×** median factor vs separately evaluating both components
Five systematic variations in sequential batches	**722.51 ms**	Graph reused throughout the parameter study
Five variations flattened into larger batches	**687.16 ms**	**1.05×** median factor vs the sequential paceval batches

These factors compare **alternative execution paths within paceval**, not paceval with optimized compiled C++. The native implementation is independently measured; its original per-run timings are included in the compact benchmark dataset (`examples/benchmark_runs.csv`). The research benefit verified here is transparent

graph reuse, correct shared-graph evaluation, measurable consolidation gains and interval diagnostics for the selected workload.

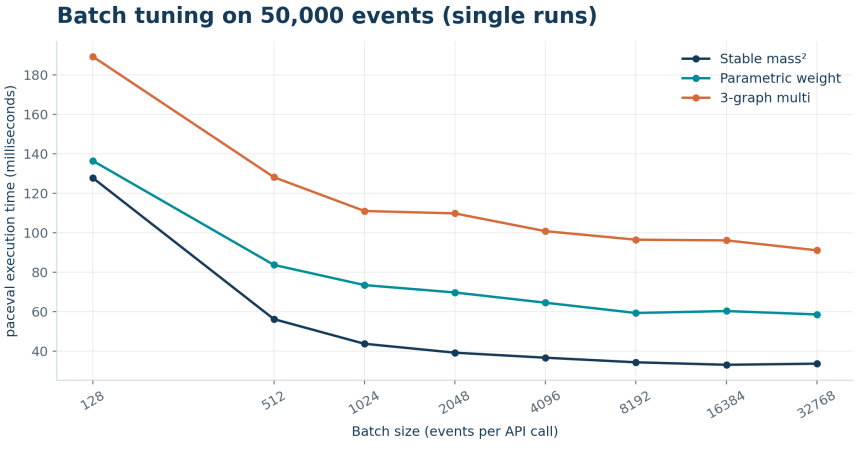


Medians and min–max of 7 process runs; 120,000 events, batch 16,384. Comparisons are between paceval execution strategies.

Paceval strategy gains

Figure 3. Median and observed ranges of seven real-runtime repetitions; all paired comparisons return the same mathematical outputs.

An exploratory, single-run batch-size sweep over **50,000** events confirms that using the batch-oriented interface efficiently matters. For the stable mass formula, the observed paceval evaluation time decreased from approximately **128 ms** with batches of 128 to **33 ms** with batches of 16,384. This exploratory sweep is not reported as a multi-run statistical estimate.



Original paceval shared library, Linux x86-64. Run-to-run variation makes these exploratory performance curves.

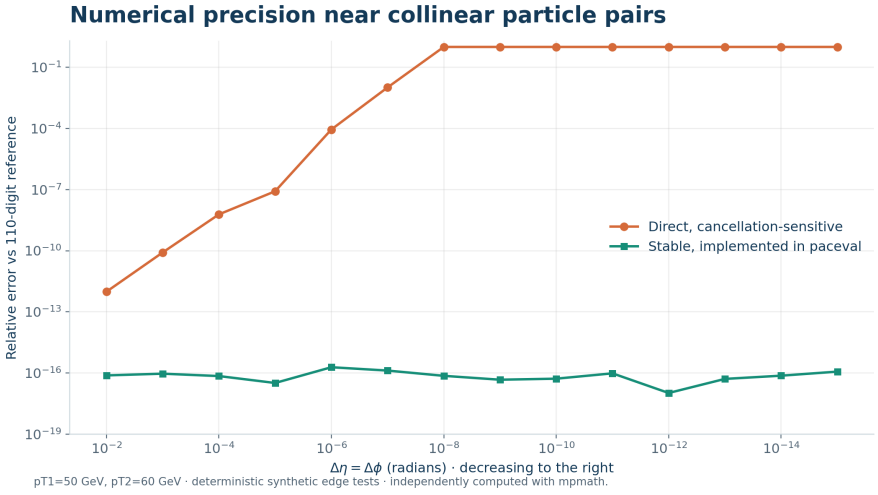
Batch-size study

Figure 4. Real-library batch-size tuning; each configuration was measured once in this exploratory sweep.

5. Numerical stability and TINC verification

The two mass formulas are analytically identical, but near-collinear pairs reveal subtractive cancellation in the direct expression. In **12,317 of 120,000** synthetic events, the direct formula exceeded **10^{-6}** relative error while the stable form remained below that threshold, when compared with an extended-precision point reference. In **5,144** events, the direct result was nonpositive despite a positive stable reference. At a deliberately artificial threshold of $m^2 > 10^{-10}\ \mathrm{GeV}^2$, eight Boolean selections differed.

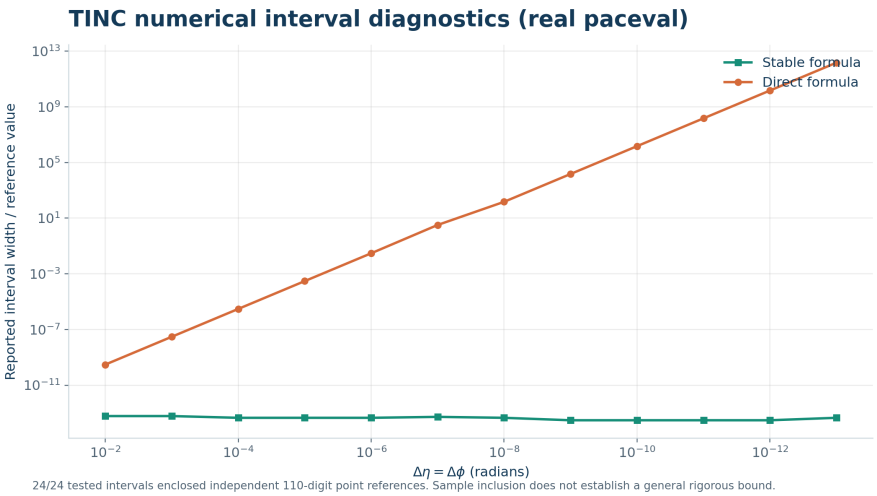
An independent `mpmath` computation at **110 decimal digits** examined 14 angles from 10^{-2} to 10^{-15} radians and found the stable expression at approximately binary64 precision across all cases. Stability originates in the chosen algebraic expression; paceval makes the expression reusable as a persistent graph.



Independent precision audit

Figure 5. Relative binary64 error against an independent 110-digit reference.

For TINC, the **actual paceval runtime** supplied intervals for 12 values per mass formula, ranging from 10^{-2} to 10^{-13} radians. **24/24 intervals** included the independently computed 110-digit reference values. The varying interval widths provide directly inspectable diagnostics of numerical sensitivity. These are empirical inclusion tests at selected point inputs, **not a formal proof of universal interval enclosure**.



TINC interval widths

Figure 6. Real paceval-reported interval width relative to the independent reference value.

6. Validation and implementation quality

- Loaded the **original library** and successfully created four normal graphs and two TINC-enabled graphs; all actual API branches executed.
- Verified **zero mismatches** beyond the defined 2×10^{-8} relative tolerance (with 10^{-28} absolute tolerance) across validated direct, stable, weight, fused, multi-graph and systematics paths.
- Passed **14** independent 110-digit formula checks and **24** 110-digit TINC interval inclusion checks.
- Compiled under GCC 14.2 and Clang 17. CTest self-tests and fixture integration tests passed.
- Exercised real-library batch sizes 1, 17, 257 and 16,384, and loaded external CSV event input.
- Completed an AddressSanitizer/UndefinedBehaviorSanitizer run with the original library, without reported diagnoses from the instrumented application.
- Included complete C++ source (`src/collider.cpp`), CMake configuration (`src/CMakeLists.txt`), original benchmark observations (`examples/benchmark_runs.csv`), independent numerical reference data (`examples/high_precision_reference.csv`), and the 110-digit checking script (`src/high_precision_reference.py`). The vendor library is not bundled; reproducing the paceval measurement requires a separately obtained matching runtime.

7. Scientific value and scaling perspectives

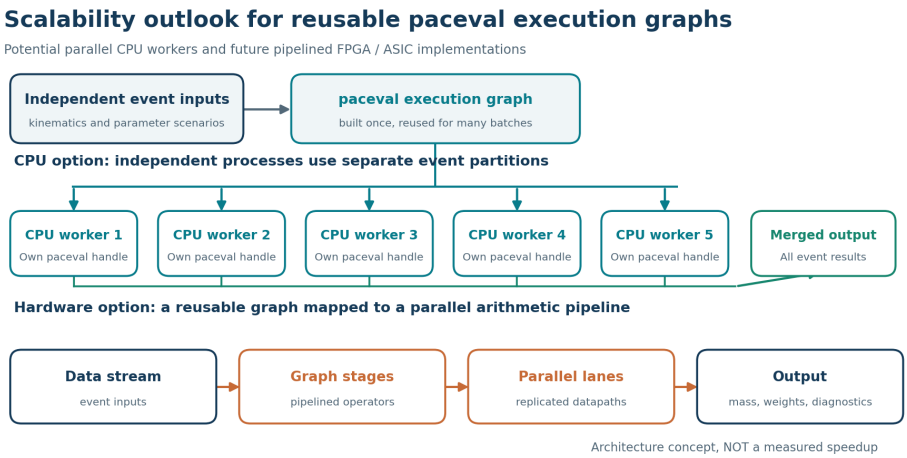
The validated implementation demonstrates **one-time creation and repeated evaluation of mathematical graphs** for collider-inspired kinematic observables, **joint evaluation of several graphs**, **fused computation of related quantities**, **repeated systematic parameter studies**, and **TINC numerical diagnostics**. All of these capabilities are exercised through the original paceval runtime. Together they provide a practical and programmable mathematical execution layer for large-event-sample research.

More CPU cores: a promising opportunity for higher total throughput

Independent collision events and many systematic-parameter scenarios can be partitioned into separate batches. This makes the workload a natural candidate for **process-level or independently managed worker parallelism**: on larger multicore CPUs or clusters, several independent workers could each reuse their own paceval graphs and process a separate event partition. With sufficient event volume and adequately provisioned cores and memory, **material and potentially significant improvements in aggregate event throughput are plausible**. The current median speed factors reported in Section 4 were measured with sequential evaluations in one process; **no multicore scaling factor was experimentally measured**. A follow-up benchmark across 1, 2, 4, 8 and more workers would establish actual scaling and saturation.

Future paceval execution graphs in FPGA / ASIC hardware

The explicit execution-graph representation also offers an attractive starting point for **pipelined, replicated mathematical kernels implemented in FPGA or ASIC technology**. A suitable architecture could process many independent event inputs concurrently, exploit carefully designed arithmetic stages and reduce repeated software scheduling overhead. Depending on graph complexity, numerical precision, memory bandwidth and pipeline replication, **substantial additional throughput increases may become possible**. These are hardware-development opportunities, not performance values extrapolated from the current CPU measurements; an actual FPGA/ASIC implementation and workload-level measurement are required to quantify them.



Potential scaling architecture for separate CPU workers and a future FPGA/ASIC graph pipeline

Figure 7. Conceptual future architecture for CPU process-level parallelism and pipelined hardware. The drawing is illustrative, not a measured speedup curve.

The next research integration would use approved physics correction formulas and documented open collider-event samples, preserving established frameworks such as ROOT as the surrounding event-analysis environment. The CSV adapter already accepts externally exported event kinematics.

8. Reproducing the included study

The GitHub package contains the companion `README.md` and this white paper at repository root. All source and build configuration files are in `src/` (`src/collider.cpp`, `src/CMakeLists.txt`, `src/high_precision_reference.py`, and `src/repeat_benchmarks.py`). The provided deterministic event fixtures and seven-run measurement CSVs reside in `examples/` (`examples/example_events.csv`, `examples/benchmark_runs.csv`), and the seven scientific illustrations in `figures/` (`figures/01_kinematics.png` through `figures/07_parallel_hardware_outlook.png`). The additional hero illustration appears in the README.

The Linux x86-64 paceval runtime is **not bundled** with the repository. On a machine with access to a compatible original paceval shared library, set its absolute path before running the following commands from the repository root:

```
export PACEVAL_LIB=/absolute/path/to/libpaceval_linux_sharedLIB.so
cmake -S src -B build -DCMAKE_BUILD_TYPE=Release -DPACEVAL_RUNTIME="$PACEVAL_LIB"
cmake --build build --parallel
ctest --test-dir build --output-on-failure
mkdir -p out
./build/collider --library "$PACEVAL_LIB" \
  --require-paceval --events 120000 --batch 16384 \
  --csv out/benchmark.csv
python3 src/high_precision_reference.py \
  --tinc-csv examples/tinc_samples.csv --strict-tinc \
  --out out/high_precision_audit.csv
python3 src/repeat_benchmarks.py --exe build/collider \
  --library "$PACEVAL_LIB" \
  --events 120000 --batch 16384 --repeats 7 --out out/repeats
```

The high-precision script requires `mpmath` (`python3 -m pip install mpmath`), whereas the C++ application itself requires only the C++17 standard library and an external original paceval shared runtime for paceval computations. The event example (`examples/example_events.csv`) accepts preselected external kinematics. The originally measured Linux library has SHA-256 `acf63a87df0ee822b467edbca887393d31ec0ea03562e1abf802d2512a5bfb12`, so reproducibility can be assessed against the same build. All results describe synthetic research demonstrations, not CERN measurements.

References

1. paceval UG, C/C++ API: creation, batch execution, multi-graph evaluation and TINC.
2. paceval UG, Mathematical expression engine and supported formula syntax.
3. ROOT, RDataFrame documentation and vector-based data analysis.
4. CERN, Open Data Portal.
5. GNU MPFR / FLINT-Arb, multiple-precision arithmetic and interval-reference frameworks for independent mathematical assessment.
6. Companion README (`README.md`), source (`src/collider.cpp`), benchmark observations (`examples/benchmark_runs.csv`), and TINC samples (`examples/tinc_samples.csv`).

Scope: Independent scientific research demonstrator using synthetic event inputs and the actual paceval shared library. No CERN or experimental-collaboration affiliation is implied.